

Data Structures & Systems Design Assessment

Submitted by: D. Rishitha Devi

Problem 1: LRU Cache Implementation

Approach

I used a hash map and a doubly linked list. The hash map provides fast access to cache entries, while the doubly linked list maintains the order of usage. When an item is accessed or updated, it is moved to the front. If the cache reaches capacity, the least recently used item is removed from the end of the list. This allows both `get()` and `put()` operations to run in $O(1)$ time.

```
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = {}

        self.head = Node(0, 0)
        self.tail = Node(0, 0)

        self.head.next = self.tail
        self.tail.prev = self.head

    def remove_node(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def add_to_front(self, node):
        node.next = self.head.next
        node.prev = self.head

        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.cache:
            return -1

        node = self.cache[key]
        self.remove_node(node)
        self.add_to_front(node)

        return node.value

    def put(self, key, value):
        if key in self.cache:
            node = self.cache[key]
            node.value = value

            self.remove_node(node)
            self.add_to_front(node)
            return

        if len(self.cache) == self.capacity:
            lru = self.tail.prev
            self.remove_node(lru)
            del self.cache[lru.key]

        new_node = Node(key, value)
        self.cache[key] = new_node
        self.add_to_front(new_node)
```

Problem 2: Event Scheduler

Approach

To check whether all events can be attended, I sort the events and look for overlaps. For calculating the minimum number of rooms, I use a min-heap that stores meeting end times. Rooms are reused whenever possible, and the maximum number of simultaneously occupied rooms gives the minimum number of rooms required.

```
import heapq

def can_attend_all(events):
    events.sort(key=lambda x: x[0])

    for i in range(1, len(events)):
        if events[i][0] < events[i - 1][1]:
            return False

    return True

def min_rooms_required(events):
    if not events:
        return 0

    events.sort(key=lambda x: x[0])

    rooms = []
    heapq.heappush(rooms, events[0][1])

    for start, end in events[1:]:
        if start >= rooms[0]:
            heapq.heappop(rooms)

        heapq.heappush(rooms, end)

    return len(rooms)
```

Final Discussion & Analysis

Time & Space Complexity

LRU Cache: $\text{get}() = O(1)$, $\text{put}() = O(1)$, Space = $O(\text{capacity})$

$\text{can_attend_all}()$: $O(n \log n)$ time, $O(1)$ extra space

$\text{min_rooms_required}()$: $O(n \log n)$ time, $O(n)$ space

Trade-offs

A hash map provides fast lookup, while a doubly linked list efficiently maintains usage order. Combining both structures enables $O(1)$ cache operations.

Future Proofing

Room identifiers such as Room A, Room B, and Room C can be stored alongside end times in the heap, allowing the scheduler to return both room assignments and room count.

Concurrency

Thread safety can be implemented using locks or read-write locks to prevent race conditions when multiple threads access or modify the cache simultaneously.